Estimation des modalités de mobilité à partir d'images

Plan de travail du groupe : étapes, organisation et planning

Polycompétence Imagerie Numérique, Université Clermont Auvergne · version du 5 novembre 2026

Ce document décrit tout le travail que nous allons réaliser d'ici la soutenance du 14 janvier 2027. Il sert de référence commune pour les cinq membres du groupe : chacun y trouve son rôle, les fichiers qu'il doit produire, le format exact de ces fichiers et les échéances. Les sections 1 à 5 donnent la vue d'ensemble et les règles communes. La section 6 détaille chaque étape technique. Les sections 7 à 11 couvrent les zones étudiées, la répartition des rôles, le planning, la soutenance et les décisions encore ouvertes.

> **À retenir**
> Les formats de fichiers de la section 5 sont le point le plus important à respecter. Tant que chacun lit et écrit exactement ces colonnes, nous pouvons tous avancer en parallèle sans attendre les autres.

1. Rappel du sujet et des rendus

Nous devons développer une application Python en ligne de commande qui estime les modalités de transport d'une zone géographique à partir d'images collaboratives de la plateforme Mapillary. À partir d'une latitude et d'une longitude, l'application télécharge les images prises dans un rayon d'environ 200 m, écarte les images inutilisables, détecte les usagers de la route avec un réseau de neurones, enregistre les comptes dans un fichier CSV, puis produit une analyse statistique et des graphiques. Nous appliquerons ensuite la solution à trois zones géographiques dont nous devrons justifier le choix.

Rendus et évaluation

- Code source commenté, déposé sur Moodle. Il fait l'objet d'une note à part entière.
- Soutenance orale de 15 minutes le jeudi 14 janvier 2027, supports en français et rendus en PDF. Elle est notée avec l'assiduité.
- Revue de projet le jeudi 26 novembre 2026 : avancement, premiers résultats, difficultés, plan de travail jusqu'à la soutenance.
- Toute source extérieure doit être citée dans le texte et dans une bibliographie. Tout usage d'IA générative pour la rédaction doit être signalé.

Catégories à détecter (minimum demandé)

Catégorie du projet	Nom de colonne	Classe(s) COCO de YOLO	Remarque
Piétons	`pietons`	`person`	Sauf les personnes sur un deux-roues (voir 6.3)
Deux-roues	`deux_roues`	`bicycle`	Vélos, avec leur cycliste
Deux-roues motorisés	`deux_roues_motorises`	`motorcycle`	Motos et scooters
Véhicules légers	`vehicules_legers`	`car`	Voitures
Bus	`bus`	`bus`	Bus et cars
Camions	`camions`	`truck`	COCO mélange utilitaires et poids lourds (voir 6.3)

Calendrier officiel des séances

Date	Horaire	Séance
Jeudi 5 novembre 2026	13h30 à 17h30	Présentation, constitution des groupes
Jeudi 12 novembre 2026	13h30 à 17h30	Travail sur le projet
Jeudi 19 novembre 2026	13h30 à 17h30	Travail sur le projet
Jeudi 26 novembre 2026	14h30 à 18h30	Revue de projet
Jeudi 10 décembre 2026	13h30 à 17h30	Travail sur le projet
Jeudi 17 décembre 2026	8h30 à 12h30 et 13h30 à 17h30	Travail sur le projet (journée complète)
Jeudi 14 janvier 2027	13h30 à 17h30	Soutenance

2. Vue d'ensemble du pipeline

Le traitement est découpé en cinq étapes indépendantes. Chaque étape lit les fichiers produits par l'étape précédente et écrit ses propres fichiers. Nous pouvons donc relancer une seule étape sans tout refaire, ce qui est essentiel car le téléchargement et le modèle vision-langage sont lents. La colonne `image_id` (identifiant Mapillary de l'image) sert de clé commune à tous les fichiers.

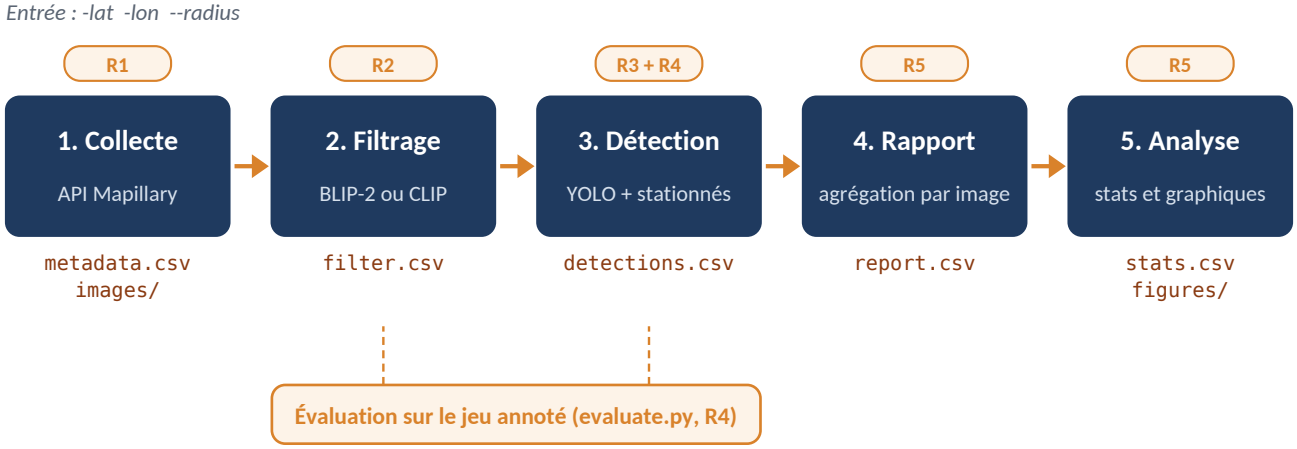


Figure 1 : les cinq étapes du pipeline, les fichiers produits et le rôle responsable (R1 à R5, voir section 8).

Étape	Ce qu'elle fait	Section
1. Collecte	Liste les images dans le rayon, sous-échantillonne, télécharge les images et leurs métadonnées.	6.1
2. Filtrage	Garde uniquement les images qui montrent une scène de circulation exploitable.	6.2
3. Détection	Détecte et classe les usagers, retire les conducteurs comptés en double, marque les véhicules stationnés.	6.3 et 6.4
4. Rapport	Compte, pour chaque image conservée, les objets en circulation de chaque catégorie.	6.5
5. Analyse	Calcule les parts modales avec leur incertitude, compare les zones, trace les graphiques.	6.6
Évaluation	Mesure la qualité du filtre, des comptes et de la détection des stationnés sur des images annotées à la main.	6.7

3. Organisation du dépôt GitHub

```
mobilite/
├── main.py               # point d'entrée : options (argparse) et enchaînement des étapes
├── config.py             # catégories, correspondance COCO, seuils, chemins par défaut
├── mapillary_client.py   # étape 1 : tuiles, rayon, sous-échantillonnage, téléchargement
├── filtering.py          # étape 2 : filtre BLIP-2 et filtre CLIP
├── detection.py          # étape 3 : YOLO, regroupement des classes, dédoublonnage
├── parked.py             # étape 3 : stratégies véhicule stationné / en circulation
├── report.py             # étape 4 : detections.csv -> report.csv
├── analysis.py           # étape 5 : statistiques et graphiques, comparaison des zones
├── evaluate.py           # comparaison avec le jeu annoté
├── annotations/          # annotations.csv (jeu annoté à la main, versionné)
├── requirements.txt      # versions figées des bibliothèques
├── .env.example          # MAPILLARY_TOKEN=MLY|...  (le vrai .env n'est jamais versionné)
├── references.md         # bibliographie tenue à jour dès le début
├── usage_ia.md           # journal des usages d'IA générative (obligatoire pour la soutenance)
└── data/                 # non versionné
    └── <zone>/
        ├── images/
        ├── metadata.csv  filter.csv  detections.csv  report.csv  stats.csv
        └── figures/
```

Règles communes

- Une branche par fonctionnalité. La fusion dans `main` passe par une pull request relue par un autre membre.
- Chaque membre crée son propre compte Mapillary et son propre token. Le token est lu dans la variable d'environnement `MAPILLARY_TOKEN` (fichier `.env`), jamais écrit dans le code.
- Le dossier `data/` et les poids des modèles (`*.pt`) sont dans le `.gitignore`. Les images partagées passent par un dossier commun (Drive), pas par Git.
- Python 3.10 ou plus, versions figées dans `requirements.txt`.
- Le code est noté : commentaires en français, une docstring par fonction (entrées, sorties, rôle), annotations de type.
- Graine aléatoire fixée partout où il y a du hasard (sous-échantillonnage, bootstrap) pour des résultats reproductibles.
- Chaque étape écrit un journal (`logging`) : nombre d'images traitées, rejetées, erreurs.
- Chacun ajoute ses sources dans `references.md` et ses usages d'IA générative dans `usage_ia.md` (qui, quel outil, pour quelle partie) au moment où il les utilise, pas à la fin.

4. Interface en ligne de commande

Les options `-h`, `-lat`, `-lon` et `-o` sont imposées par le sujet. Les autres options nous servent à développer, à tester et à comparer les variantes. Le module `argparse` accepte sans problème les options à un tiret de plusieurs lettres comme `-lat`.

Option	Défaut	Rôle
`-h`		Affiche l'aide
`-lat`, `-lon`	obligatoires	Centre de la zone étudiée
`-o`	`report.csv`	Fichier CSV de résultats
`--radius`	200	Rayon en mètres
`--zone`	déduit de lat/lon	Nom du dossier de travail dans `data/`
`--spacing`	15	Distance minimale en mètres entre deux images gardées d'une même séquence
`--max-images`	aucun	Limite le nombre d'images (tests rapides)
`--filter`	à fixer (6.2)	`blip2`, `clip` ou `none`
`--detector`	`yolo11s.pt`	Poids YOLO à utiliser
`--conf`, `--imgsz`	à régler (6.3)	Seuil de confiance et taille d'entrée de YOLO
`--keep-parked`	désactivé	Compte aussi les véhicules stationnés (pour mesurer l'effet du tri)
`--skip-download`	désactivé	Réutilise les images déjà téléchargées
`--steps`	toutes	Étapes à exécuter, par exemple `--steps detect,report`
`--device`	`cuda`	`cuda` ou `cpu`

```
# exemple : carrefour des pistes, rayon 200 m
python main.py -lat 45.800413 -lon 3.108813 -o report.csv --zone carrefour_pistes

# relancer uniquement la détection et le rapport avec un autre modèle
python main.py -lat 45.800413 -lon 3.108813 -o report.csv --zone carrefour_pistes \
               --steps detect,report --detector yolo11m.pt

# analyse comparée des trois zones (script séparé)
python analysis.py --zones data/zone_a data/zone_b data/zone_c --out figures/
```

5. Contrats de données (formats des fichiers)

Ces formats sont à valider ensemble lors de la première séance, puis à ne plus modifier sans prévenir tout le groupe. Tous les fichiers sont des CSV avec séparateur virgule, encodés en UTF-8, une ligne d'en-tête.

metadata.csv (étape 1, une ligne par image candidate)

Colonne	Type	Description
`image_id`	texte	Identifiant Mapillary, clé commune à tous les fichiers
`sequence_id`	texte	Séquence (trajet) à laquelle appartient l'image
`lat`, `lon`	réel	Position de l'image
`distance_m`	réel	Distance au centre de la zone
`captured_at`	date	Date et heure de prise de vue
`compass_angle`	réel	Orientation de la caméra en degrés
`is_pano`	booléen	Image panoramique 360°
`selected`	booléen	Gardée après sous-échantillonnage et exclusion des panoramas
`path`	texte	Chemin du fichier image (vide si non téléchargée)

filter.csv (étape 2, une ligne par image téléchargée)

Colonne	Type	Description
`image_id`	texte	
`method`	texte	`blip2` ou `clip`
`answer`	texte	Réponse brute de BLIP-2 (vide pour CLIP)
`score`	réel	Score de CLIP entre 0 et 1 (vide pour BLIP-2)
`keep`	booléen	Décision finale

detections.csv (étape 3, une ligne par objet détecté)

Colonne	Type	Description
`image_id`	texte	
`coco_class`	texte	Classe renvoyée par YOLO
`category`	texte	Catégorie du projet (noms de colonnes de la section 1)
`conf`	réel	Confiance de la détection
`x1, y1, x2, y2`	réel	Boîte englobante en pixels
`is_rider`	booléen	Personne sur un deux-roues, non comptée comme piéton
`parked`	booléen	Véhicule jugé stationné (vide pour les piétons)
`parked_method`	texte	Stratégie utilisée pour décider (6.4)

report.csv (étape 4, format demandé par le sujet, une ligne par image conservée)

Colonne	Type	Description
`image`	texte	Nom du fichier image, par exemple `123456789.jpg`
`pietons` … `camions`	entier	Nombre d'objets en circulation de chaque catégorie (six colonnes)

stats.csv (étape 5, une ligne par zone et par catégorie)

Colonne	Type	Description
`zone`, `category`	texte	
`count`	entier	Nombre total d'objets
`share`, `share_low`, `share_high`	réel	Part modale estimée et intervalle de confiance à 95 %
`presence_rate`	réel	Part des images contenant au moins un objet de la catégorie
`mean_per_image`	réel	Nombre moyen d'objets par image

6. Étapes détaillées

Pour chaque étape : l'objectif, le travail à réaliser, le fichier produit et les points d'attention.

6.1 Collecte des images Mapillary (R1)

Objectif : obtenir un ensemble d'images représentatif de la zone, sans doublons excessifs, avec leurs métadonnées. Le fichier `test_student.py` fourni est notre point de départ (l'annexe A liste ce qu'il faut corriger).

1. Convertir le rayon en boîte englobante : Δlat = r / 111 320 et Δlon = r / (111 320 × cos(lat)). Pour 200 m à Clermont-Ferrand, cela donne environ 0,0018° en latitude et 0,0026° en longitude. Le script fourni utilise 0,003° dans les deux sens, soit un carré d'environ 330 m sur 230 m de demi-côté.
2. Lister les tuiles au zoom 14 avec `mercantile` et lire la couche `image` avec `vt2geojson`, comme dans le script fourni.
3. Garder uniquement les points dont la distance au centre, calculée avec la formule de haversine, est inférieure ou égale au rayon.
4. Sous-échantillonner : dans chaque séquence, trier les images par date et garder une image seulement si elle est à au moins `--spacing` mètres de la dernière gardée. Sans cela, le même véhicule est compté sur plusieurs images successives.
5. Interroger l'API Graph pour chaque image gardée avec `fields=thumb_2048_url,captured_at,compass_angle,is_pano`, exclure les panoramas, puis télécharger la vignette dans `images/<image_id>.jpg`.
6. Écrire `metadata.csv` avec toutes les images candidates, y compris celles non retenues (colonne `selected`). Cela nous donne le nombre d'images disponibles par zone pour la soutenance.

Robustesse attendue

- Délai maximal (`timeout`) sur chaque requête et trois tentatives avec attente en cas d'échec.
- Une image déjà présente sur le disque n'est pas téléchargée à nouveau : la collecte peut être interrompue et relancée.
- Une réponse sans `thumb_2048_url` est journalisée et ignorée au lieu de faire planter le programme.
- Une fonction `count_images(lat, lon, radius)` qui compte les images disponibles sans rien télécharger. Elle sert à vérifier la densité d'images avant de choisir une zone (section 7).

> **Pour démarrer vite**
> R1 télécharge dès aujourd'hui, avec le script fourni, un jeu de démarrage d'environ 200 images du carrefour des pistes et le dépose dans le dossier partagé. R2, R3 et R4 peuvent ainsi commencer sans attendre la collecte définitive.

6.2 Filtrage des images avec un modèle vision-langage (R2)

Objectif : garder les images qui montrent une scène de circulation exploitable et écarter les autres. Nous comparons deux méthodes, BLIP-2 demandé par le sujet et CLIP en alternative plus légère, puis nous retenons la meilleure sur des critères mesurés.

Définition d'une image conservée (à valider en groupe)

Une image est conservée si elle montre, depuis l'espace public, une voie ouverte à la circulation : route, rue, piste cyclable ou trottoir longeant une chaussée. Les vues prises depuis un trottoir sont gardées car elles montrent aussi les piétons et les cyclistes. Sont écartés : les parkings (figure 2 du sujet), les vues où l'intérieur du véhicule domine, les images trop floues ou trop sombres, les vues sans chaussée (façade, champ, chantier fermé).

Méthode A : BLIP-2

- Modèle `Salesforce/blip2-opt-2.7b` chargé en demi-précision (`torch_dtype=torch.float16`) sur GPU, soit environ 8 Go de mémoire graphique. Si le GPU est plus petit, chargement en 8 bits.
- Tester au moins trois formulations de question, par exemple : « Question: Is this a street scene viewed from a road? Answer yes or no. Answer: ».
- Règle de décision : la réponse nettoyée (minuscules, espaces retirés) commence par « yes ». La réponse brute est toujours enregistrée.
- Variante à tester : une seconde question de rejet (« Is this a parking lot? ») qui écarte l'image si la réponse est « yes ».

Méthode B : CLIP en classification sans entraînement

- Modèle `openai/clip-vit-large-patch14` (ou la version `base-patch32` si le temps de calcul est trop long).
- Descriptions positives, par exemple « a photo of a street with traffic taken from the road », « a city street seen from the sidewalk ». Descriptions négatives, par exemple « a parking lot full of parked cars », « the inside of a car », « a building facade », « a blurry photo ».
- Score = somme des probabilités des descriptions positives. L'image est gardée si le score dépasse un seuil choisi sur la partie réglage du jeu annoté (6.7).

Comparaison et choix

Les deux méthodes sont évaluées sur la partie test du jeu annoté. Nous remplirons le tableau suivant, qui ira tel quel dans la soutenance :

Méthode	Précision	Rappel	F1	Temps par image	Mémoire GPU
BLIP-2 (meilleure question)					
BLIP-2 + question de rejet					
CLIP (seuil réglé)					

Point d'attention : ne jamais régler un seuil ou choisir une question sur les images qui servent à l'évaluation finale, sinon les chiffres présentés seront trop optimistes.

6.3 Détection des usagers avec YOLO (R3)

- Modèles YOLO11 pré-entraînés sur COCO avec la bibliothèque `ultralytics`. `yolo11n.pt` pour développer, puis comparaison de `yolo11s`, `yolo11m` et `yolo11l` sur le jeu annoté pour choisir le meilleur compromis entre temps de calcul et erreur de comptage.
- Les vignettes font 2048 pixels de large et les usagers lointains sont petits : tester `imgsz=1280` en plus de la valeur par défaut (640). Le seuil `conf` est à régler entre 0,25 et 0,5 sur la partie réglage.

- Traitement par lots sur GPU. Seules les six classes COCO de la section 1 sont conservées, les autres sont ignorées.
- Sauvegarder les images annotées (boîtes dessinées) pour un échantillon : elles serviront d'exemples dans la soutenance.

Ne pas compter deux fois les cyclistes et motards

Un cycliste est détecté à la fois comme `person` et comme `bicycle`. Pour chaque personne P et chaque deux-roues V de la même image, nous calculons la part de la boîte de P couverte par V (aire de l'intersection divisée par l'aire de P). Si cette part dépasse un seuil (0,3 au départ, à régler) et que le centre horizontal de P tombe dans V, la personne est marquée `is_rider` et n'est pas comptée comme piéton. Chaque deux-roues ne peut être associé qu'à une seule personne pour un vélo, deux pour une moto.

Camions et véhicules utilitaires

La classe `truck` de COCO regroupe pick-up, fourgonnettes et poids lourds. En extension, nous pourrons distinguer utilitaires et poids lourds avec CLIP sur le recadrage du véhicule (« a delivery van » contre « a heavy truck »). La décision d'ajouter ou non cette catégorie est à prendre ensemble (section 11).

6.4 Véhicules stationnés ou en circulation (R4)

C'est la principale difficulté signalée par le sujet : un véhicule garé ne témoigne pas d'un déplacement. Nous testons plusieurs stratégies, de la plus simple à la plus coûteuse, et nous mesurons l'apport de chacune.

Stratégie	Principe	Coût
1. Filtre de l'image	Les images de parking sont déjà écartées à l'étape 2.	Nul
2. Règle géométrique	Hypothèse à vérifier : un véhicule garé apparaît souvent sur les bords latéraux de l'image et de profil (boîte nettement plus large que haute), alors qu'un véhicule qui circule devant la caméra est vu de dos, plutôt au centre.	Très faible
3. Question sur le recadrage	Découper la boîte avec une marge de 20 % et demander à CLIP « a parked car on the side of the street » contre « a car driving on the road » (ou poser la question à BLIP-2).	Moyen
4. Segmentation de la chaussée (piste avancée)	Modèle de segmentation sémantique entraîné sur des scènes urbaines (par exemple SegFormer sur Cityscapes ou Mapillary Vistas) pour savoir si le véhicule est sur une voie de circulation. Limite : les places de stationnement le long de la chaussée ne sont pas toujours distinguées.	Élevé

- Les stratégies s'appliquent aux voitures, bus, camions et aussi aux vélos et motos garés. Les piétons ne sont pas concernés.
- Par défaut, le rapport ne compte que les objets avec `parked` faux. L'option `--keep-parked` permet de mesurer l'effet du tri sur les parts modales.
- Chaque stratégie est évaluée sur le jeu annoté, où les véhicules stationnés sont comptés à part (6.7).

6.5 Génération du rapport CSV (R5)

- Une ligne par image conservée par le filtre, au format de la section 5. Les images rejetées restent tracées dans `filter.csv`.
- Un objet est compté s'il n'est pas un conducteur (`is_rider` faux) et n'est pas stationné (`parked` faux).
- Une image conservée sans aucune détection apparaît quand même, avec des zéros : elle compte dans les statistiques.

6.6 Analyse statistique et graphiques (R5)

Indicateurs calculés pour chaque zone

Indicateur	Définition	Pourquoi
Part modale estimée	Objets de la catégorie divisés par le total des objets	Indicateur principal du projet
Taux de présence	Part des images avec au moins un objet de la catégorie	Moins sensible aux images très chargées
Moyenne par image	Nombre moyen d'objets par image, avec écart-type	Intensité du trafic visible
Images disponibles et conservées	Issus de `metadata.csv` et `filter.csv`	Fiabilité de l'estimation

Incertitude et comparaison des zones

- Intervalles de confiance à 95 % par bootstrap (1 000 tirages). Nous rééchantillonnons les séquences et non les images, car les images d'une même séquence sont corrélées (même trajet, même moment).
- Comparaison des zones par un test du khi-deux sur le tableau zone × catégorie, interprété avec prudence car les observations ne sont pas indépendantes. Les intervalles de confiance restent l'argument principal.

Graphiques prévus

- Parts modales par zone : barres empilées à 100 %, et une version en barres groupées avec intervalles de confiance.
- Taux de présence par catégorie et par zone.
- Carte des images de chaque zone (longitude, latitude), colorées selon qu'elles sont conservées ou rejetées, avec le cercle du rayon.
- Répartition des dates de prise de vue par zone (les images peuvent avoir plusieurs années d'écart).
- Effet de chaque étape : parts modales sans filtre, avec filtre, puis avec filtre et sans les stationnés.
- Planche d'exemples : détections réussies, erreurs typiques, images rejetées par le filtre.

Biais à discuter en soutenance

Les contributeurs de Mapillary circulent surtout en voiture avec une caméra embarquée : le point de vue favorise les véhicules situés sur la chaussée. Les images ne sont prises ni aux mêmes heures, ni aux mêmes saisons, ni aux mêmes années selon les zones, et la densité d'images varie beaucoup. En option, nous pourrons confronter nos parts modales aux données du recensement de l'INSEE sur le mode de transport domicile-travail, en rappelant qu'il ne s'agit pas de la même grandeur.

6.7 Évaluation sur un jeu annoté (R4, annotation par tous)

Sans vérité terrain, nous ne pouvons ni régler les seuils ni défendre nos résultats. Nous annotons donc nous-mêmes un petit jeu d'images.

- 150 images tirées au hasard dans les zones étudiées, dont le carrefour des pistes, soit 30 images par membre.
- Pour chaque image : l'image doit-elle être conservée (selon la définition de 6.2), le nombre d'objets en circulation par catégorie, le nombre de véhicules stationnés.
- Format : `annotations/annotations.csv` avec les colonnes `image_id, keep, pietons, …, camions, stationnes, annotateur, commentaire`.
- 20 images sont annotées par deux personnes différentes pour vérifier que nous appliquons les mêmes règles. Les désaccords servent à préciser les consignes.

- Séparation fixe : 75 images pour régler les seuils et les questions, 75 images pour l'évaluation finale, jamais utilisées pour régler.

Mesures

Élément évalué	Mesures
Filtre (BLIP-2, CLIP)	Précision, rappel, F1 sur la décision « conserver »
Comptes par catégorie	Erreur absolue moyenne par image et biais moyen (sur-comptage ou sous-comptage)
Véhicules stationnés	Part des véhicules correctement classés stationnés ou en circulation, pour chaque stratégie
Parts modales	Parts estimées par le pipeline comparées aux parts calculées sur les annotations

Une étude d'ablation (pipeline complet, puis sans filtre, sans dédoublonnage, sans tri des stationnés) montrera l'apport de chaque étape.

7. Choix des trois zones géographiques

Le sujet nous laisse libres, à condition de justifier le choix. R5 propose les zones, le groupe décide le 10 décembre au plus tard.

Critères

- Nombre suffisant d'images dans le rayon, vérifié avec `count_images` avant toute analyse.
- Contraste attendu entre les zones, pour que la comparaison ait un sens.
- Au moins une zone où le résultat attendu est connu, ce qui sert de contrôle de cohérence de la méthode.

Deux options à discuter

Option	Zones possibles	Intérêt
A. Gradient urbain	Centre de Clermont-Ferrand (place de Jaude), carrefour des pistes, une commune rurale du Puy-de-Dôme	Terrain connu, comparaison urbain, périurbain, rural. Risque : peu d'images en zone rurale.
B. Trois pays	Clermont-Ferrand, Amsterdam ou Copenhague, une ville d'Europe du Sud	Amsterdam ou Copenhague doivent faire ressortir une forte part du vélo : c'est un test de la méthode. Une ville d'Europe du Sud permet de tester les deux-roues motorisés.

8. Répartition des rôles

Nous sommes cinq. Les rôles sont génériques (R1 à R5) et seront attribués en séance. Chaque rôle possède un ou deux modules et en est responsable de bout en bout : code, tests, commentaires et partie correspondante de la soutenance.

Rôle	Modules	Livrables principaux	Échéances clés

R1 Collecte et intégration	`mapillary_client.py`
	`main.py`
	`config.py`
		Jeu de démarrage, collecte robuste, `count_images`, pipeline complet lancé par `main.py`, README, gestion du dépôt	5/11 jeu de démarrage, 12/11 collecte, 19/11 intégration

R2 Filtrage VLM	`filtering.py`	BLIP-2 et CLIP opérationnels, comparaison chiffrée, choix argumenté. Référent bibliographie et `usage_ia.md`	19/11 prototypes, 10/12 choix final
R3 Détection	`detection.py`	YOLO, regroupement des classes, dédoublonnage des conducteurs, choix du modèle et des réglages, images annotées	12/11 version simple, 10/12 version finale
R4 Stationnés et évaluation	`parked.py` `evaluate.py`	Protocole d'annotation, jeu annoté, stratégies de tri des stationnés, toutes les mesures de qualité	12/11 protocole, 19/11 jeu annoté, 10/12 évaluation
R5 Rapport, analyse, zones	`report.py` `analysis.py`	report.csv, stats.csv, bootstrap, graphiques, proposition des zones, maquette des supports de soutenance	19/11 premiers graphiques, 17/12 résultats finaux

Tâches partagées

- Tout le monde annote 30 images du jeu d'évaluation.
- Tout le monde relit au moins une pull request d'un autre membre chaque semaine.
- Chacun prépare les diapositives de sa partie. R5 assemble les supports et vérifie la cohérence.

9. Planning

Date	Objectifs	Résultat attendu en fin de séance
Jeu. 5/11	Attribuer les rôles, créer le dépôt GitHub, créer les comptes Mapillary, valider les formats de la section 5	Dépôt en place, jeu de démarrage partagé
Jeu. 12/11	Collecte avec rayon et sous-échantillonnage, YOLO simple avec regroupement, protocole d'annotation figé, `report.py` simple	Un premier `report.csv` (sans filtre) sur le carrefour des pistes
Jeu. 19/11	Pipeline intégré dans `main.py`, prototypes BLIP-2 et CLIP, jeu annoté terminé, dédoublonnage, premiers graphiques	Premiers résultats chiffrés et premières mesures du filtre
Mer. 25/11	Préparer les diapositives de la revue de projet	Supports prêts
Jeu. 26/11	Revue de projet : état, premiers résultats, comparaison préliminaire des filtres, difficultés, plan	Retours de l'enseignant notés dans le dépôt
Du 27/11 au 9/12	Intégrer les retours, stratégies pour les stationnés, mesures sur la partie réglage	Variantes prêtes à comparer
Jeu. 10/12	Choix définitifs : filtre, modèle YOLO, réglages, stratégie stationnés, trois zones. Évaluation finale	Pipeline figé, tableaux d'évaluation remplis
Jeu. 17/12	Exécution sur les trois zones, statistiques et graphiques finaux, gel du code le soir, plan des diapositives	Tous les résultats de la soutenance
Jusqu'au 13/01	Diapositives finales, commentaires et nettoyage du code, répétition chronométrée, dépôt Moodle (code et PDF)	Tout est rendu
Jeu. 14/01	Soutenance	

10. Plan de la soutenance (15 minutes)

Les six parties sont imposées par le sujet. La répartition du temps est une proposition.

Partie imposée	Contenu	Durée
Contexte	Enjeux de la mobilité, coût des enquêtes classiques, Mapillary comme source d'images	1 min 30
Objectifs	Hypothèse de travail, catégories visées, ce que nous mesurons	1 min
Solution proposée	Pipeline, filtre (BLIP-2 contre CLIP), détection, dédoublonnage, stationnés, avec exemples visuels	5 min
Évaluation sur des zones pertinentes	Résultats sur le jeu annoté, puis résultats et comparaison des trois zones avec justification du choix	4 min 30
Difficultés et perspectives	Stationnés, biais de Mapillary, densité d'images, pistes d'amélioration	2 min
Conclusion	L'hypothèse est-elle vérifiée, et dans quelles limites	1 min

Sources à citer au minimum

- Documentation de l'API Mapillary (tuiles vectorielles et API Graph).
- Documentation d'Ultralytics et page du modèle YOLO utilisé. Jeu de données COCO (Lin et al., 2014).
- Fiche du modèle BLIP-2 sur Hugging Face, documentation de Transformers, article BLIP-2 (Li et al., 2023).
- Fiche du modèle CLIP et article CLIP (Radford et al., 2021).
- Une diapositive dédiée à l'usage des outils d'IA générative, construite à partir de `usage_ia.md`.

11. Décisions à prendre ensemble

Décision	Quand	Qui propose
Validation des formats de fichiers (section 5)	5/11	R1
Définition exacte d'une image conservée (6.2)	12/11	R2 et R4
Valeur de l'espacement du sous-échantillonnage	12/11	R1
Traitement des panoramas (exclusion ou découpage)	12/11	R1
Séparer ou non utilitaires et poids lourds	19/11	R3
Seuils : confiance YOLO, filtre, dédoublonnage	10/12, sur la partie réglage	R2, R3
Stratégie retenue pour les stationnés	10/12	R4
Les trois zones	10/12	R5

Annexe A. Points à corriger dans test_student.py

Problème	Correction prévue
Zone carrée de ±0,003° : environ 330 m en latitude et 230 m en longitude, pas un cercle de 200 m	Boîte calculée à partir du rayon, puis filtrage par distance de haversine (6.1)
Token écrit dans le code	Lecture de `MAPILLARY_TOKEN` dans l'environnement
Aucune gestion d'erreur : un token invalide ou une image sans `thumb_2048_url` fait planter le script	Délais, tentatives, journalisation, image ignorée en cas d'échec
La variable `data` est réutilisée pour la réponse de l'API image	Noms distincts (`tile_data`, `image_info`)
Toutes les images d'une séquence sont téléchargées	Sous-échantillonnage par distance (`--spacing`)
Le dossier `output` est vidé à chaque exécution et le script tente de supprimer les dossiers du répertoire courant	Écriture dans `data/<zone>/images/`, aucune suppression
Imports inutiles : `mapbox_vector_tile`, `geopy`, `pdb`, `json`	Supprimés
Commentaire de la boîte englobante inexact (ordre des coordonnées)	Commentaire corrigé : ouest, sud, est, nord

Annexe B. Extraits de code de référence

Boîte englobante et distance de haversine (R1)

```
import math

R_TERRE = 6_371_000  # rayon moyen de la Terre en mètres


def bbox_from_radius(lat: float, lon: float,
                     radius_m: float) -> tuple[float, float, float, float]:
    """Retourne (ouest, sud, est, nord) de la boîte qui contient le cercle de rayon radius_m."""
    dlat = radius_m / 111_320
    dlon = radius_m / (111_320 * math.cos(math.radians(lat)))
    return lon - dlon, lat - dlat, lon + dlon, lat + dlat


def haversine_m(lat1: float, lon1: float, lat2: float, lon2: float) -> float:
    """Distance en mètres entre deux points GPS."""
    p1, p2 = math.radians(lat1), math.radians(lat2)
    dp, dl = p2 - p1, math.radians(lon2 - lon1)
    a = math.sin(dp / 2) ** 2 + math.cos(p1) * math.cos(p2) * math.sin(dl / 2) ** 2
    return 2 * R_TERRE * math.asin(math.sqrt(a))
```

Correspondance des classes et dédoublonnage des conducteurs (R3)

```
COCO_TO_CATEGORY = {
    "person": "pietons",
    "bicycle": "deux_roues",
    "motorcycle": "deux_roues_motorises",
    "car": "vehicules_legers",
    "bus": "bus",
    "truck": "camions",
}


def part_couverte(person_box, vehicle_box) -> float:
    """Part de la boîte de la personne couverte par la boîte du deux-roues (entre 0 et 1)."""
    x1, y1 = max(person_box[0], vehicle_box[0]), max(person_box[1], vehicle_box[1])
    x2, y2 = min(person_box[2], vehicle_box[2]), min(person_box[3], vehicle_box[3])
    inter = max(0, x2 - x1) * max(0, y2 - y1)
    aire = (person_box[2] - person_box[0]) * (person_box[3] - person_box[1])
    return inter / aire if aire > 0 else 0.0
```

Ce document a été rédigé avec l'aide d'un outil d'IA générative (Claude, Anthropic) à partir du sujet du projet, puis relu par le groupe. Cet usage est consigné dans usage_ia.md.